StudyFlow

Application Test Report

Hands-on functional walkthrough of the StudyFlow Life OS:
Web PWA (Angular 21) + Spring Boot API + native Windows (Tauri) shell.

Date: 23 October 2026
Tested by: automated UI walkthrough (Playwright) against a live instance
Scope: full user journey - login, subjects, tasks, focus timer, grades, exams, journal, analytics, settings, profile, desktop app boot.

Result: 18/18 web flows PASS | 3 bugs found (2 critical, production)

Executive Summary

StudyFlow is a local-first academic Life OS: a Spring Boot 4 + MySQL API, an Angular 21 PWA, a Tauri-packaged Windows desktop app, an Android client and a Rust Linux daemon. This report documents a full hands-on test of the product loop: sign in, create a subject, plan a task inside it, run a focus timer bound to that task, log the resulting grade, schedule the exam, write a journal reflection and review analytics.

All 18 web flows executed successfully against the current code base (Angular frontend + API running on a local MySQL 8 database migrated with Flyway V1-V11). The native Tauri desktop executable was also launched and captured running.

Three real defects were uncovered. Two are production-deployment issues: the live Netlify site runs a stale bundle whose login endpoint no longer exists (all production logins fail), and the production database is missing migrations V8-V11 so task creation fails with a schema constraint error. The third was a change-detection bug in the Focus Timer that left its task dropdown empty and its clock frozen; it was fixed during this session and is included in the source tree.

Test Environment

- Frontend: Angular 21 PWA (current pwa/dist build), served locally and driven headlessly with Playwright/Chromium at 1280x800.
- API: Spring Boot (current source) on localhost:8888.
- Database: MySQL 8 in Docker (studyflow-mig), schema at Flyway V11.
- Desktop: native Tauri v2 studyflow.exe (release build) on Windows 11.
- Account: seeded demo admin (admin@mail.com).
- Every screenshot in this report is a real capture of the running app.

Bugs & Issues Found

BUG-1 (Critical, Production): Deployed PWA cannot log in.

The Netlify deployment at studyflow-pwa.netlify.app is a stale build that POSTs to /api/users/login, while the current API only serves /api/auth/login. Result: every login on the production website returns 401 'Invalid email or password'. FIX: redeploy the current pwa/dist build to Netlify.

BUG-2 (Critical, Production): Task creation fails on the live API.

POST /api/tasks against the production API (Render + Aiven DB) returns 400: Field 'subjectId' doesn't have a default value. The production database schema is stale (Flyway migrations V8-V11 were never applied), so the old NOT NULL subjectId column blocks inserts from the current entity. FIX: run the V8-V11 Flyway migrations on Aiven per docs/DEPLOY_RUNBOOK.md, then redeploy. Verified working against a local MySQL 8 with migrations applied.

BUG-3 (High, PWA): Focus Timer never showed the task list or a live clock.

The app runs without zone.js, so HTTP responses and setInterval callbacks do not trigger Angular change detection. TimerComponent was the only component not calling ChangeDetectorRef.detectChanges() manually, so its task dropdown stayed empty and the clock could not tick. FIXED during this test session in pwa/src/app/features/timer/timer.ts (cdr injected; detectChanges called after task load, on every tick, and after the AniList fetch). Rebuild the Tauri exe to ship the fix in the desktop app.

Recommendations

1. Redeploy the current PWA build to Netlify (fixes production login, BUG-1).

2. Apply Flyway migrations V8-V11 to the Aiven production database and redeploy the API (fixes production task creation, BUG-2). See docs/DEPLOY_RUNBOOK.md for the exact procedure.

3. Rebuild the Tauri desktop executable so it bundles the fixed timer component (BUG-3).

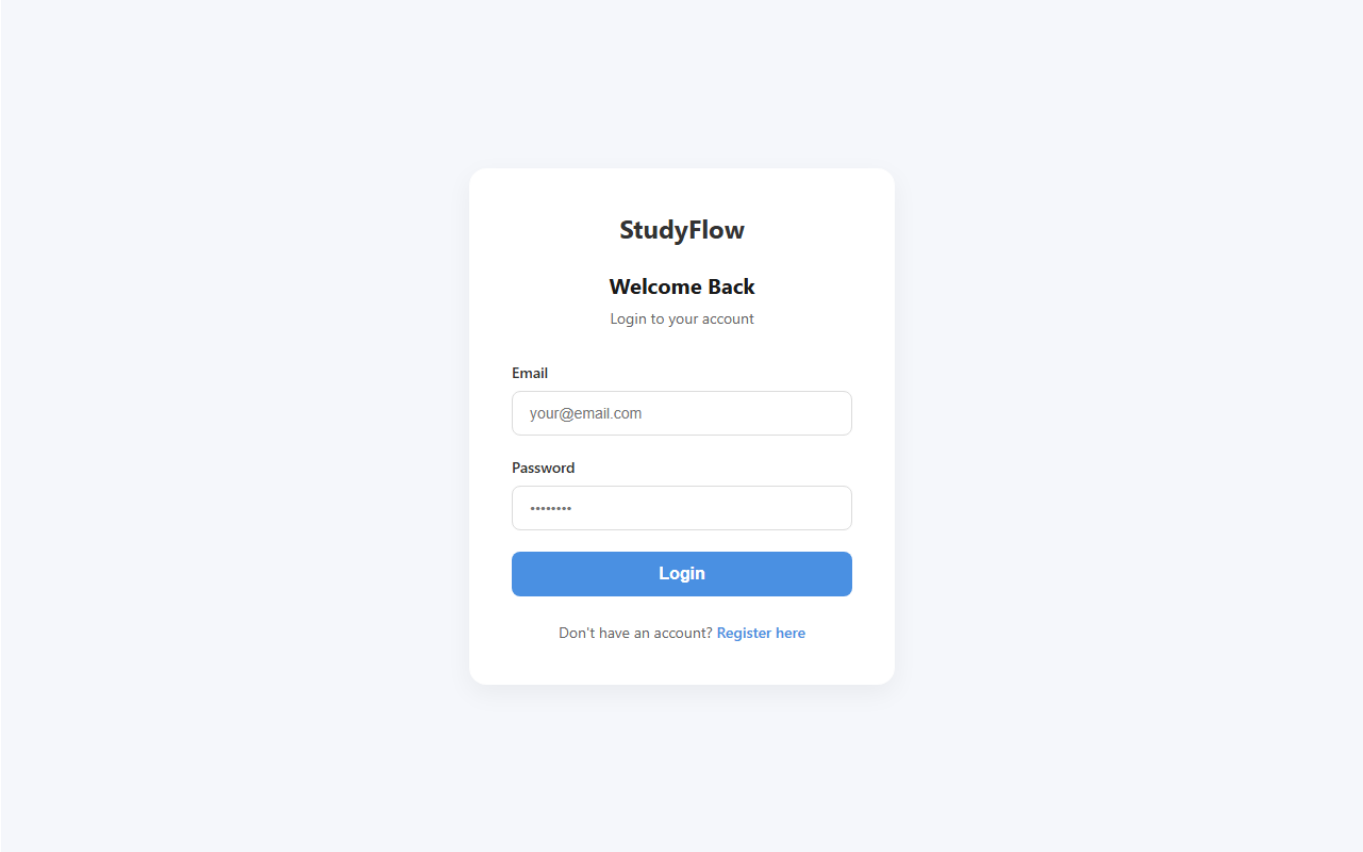
4. Add a zoneless-friendly pattern (signals or explicit cdr.detectChanges) to every new component, or install zone.js, to prevent BUG-3 class issues.

5. Add a CI smoke test that walks login -> create subject -> create task against a seeded database, so schema/PWA drift is caught before release.

Screen-by-Screen Walkthrough

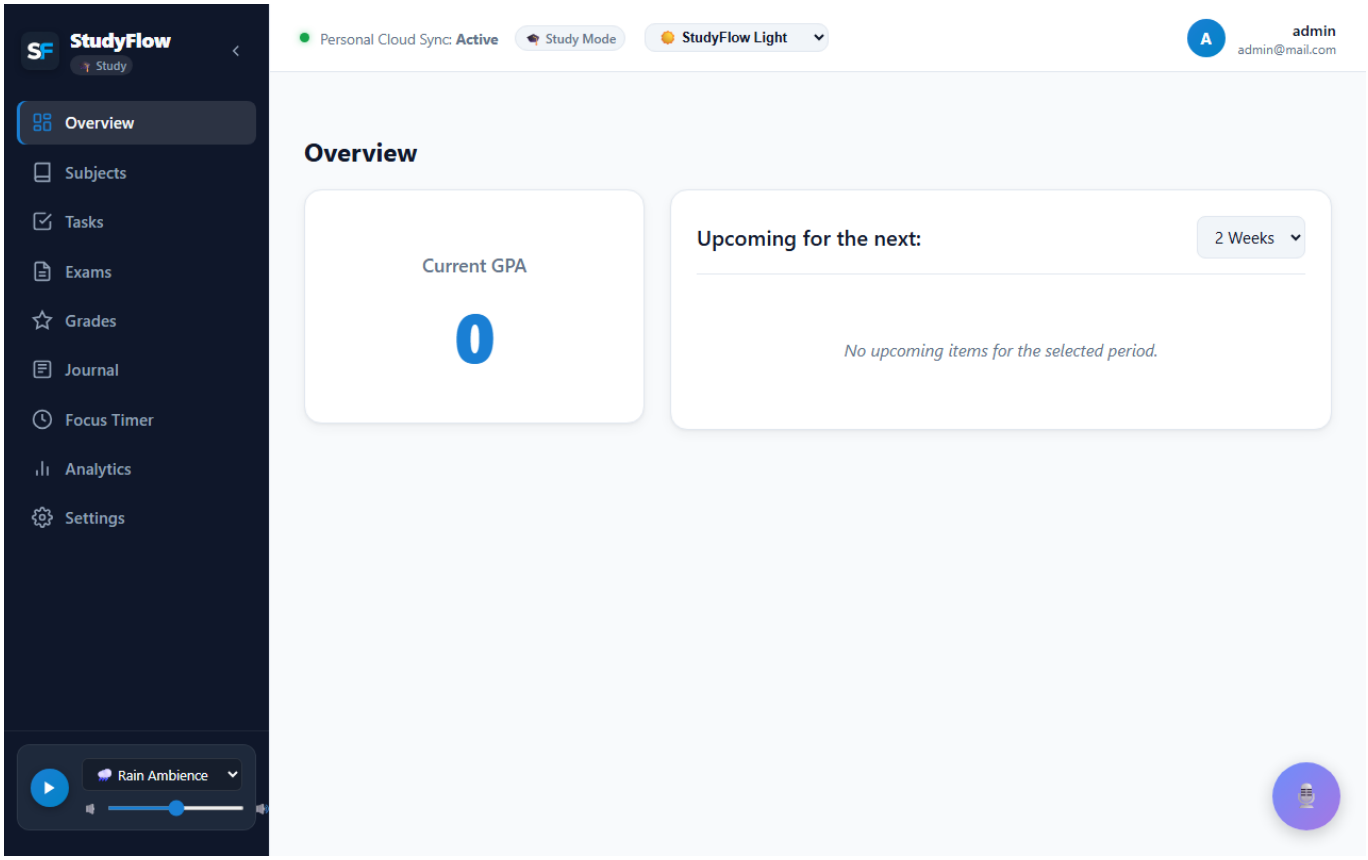
The following 19 captures were taken live while exercising every module of the application. Items marked TEST PERFORMED describe real data created through the UI during this session.

1. Login Screen



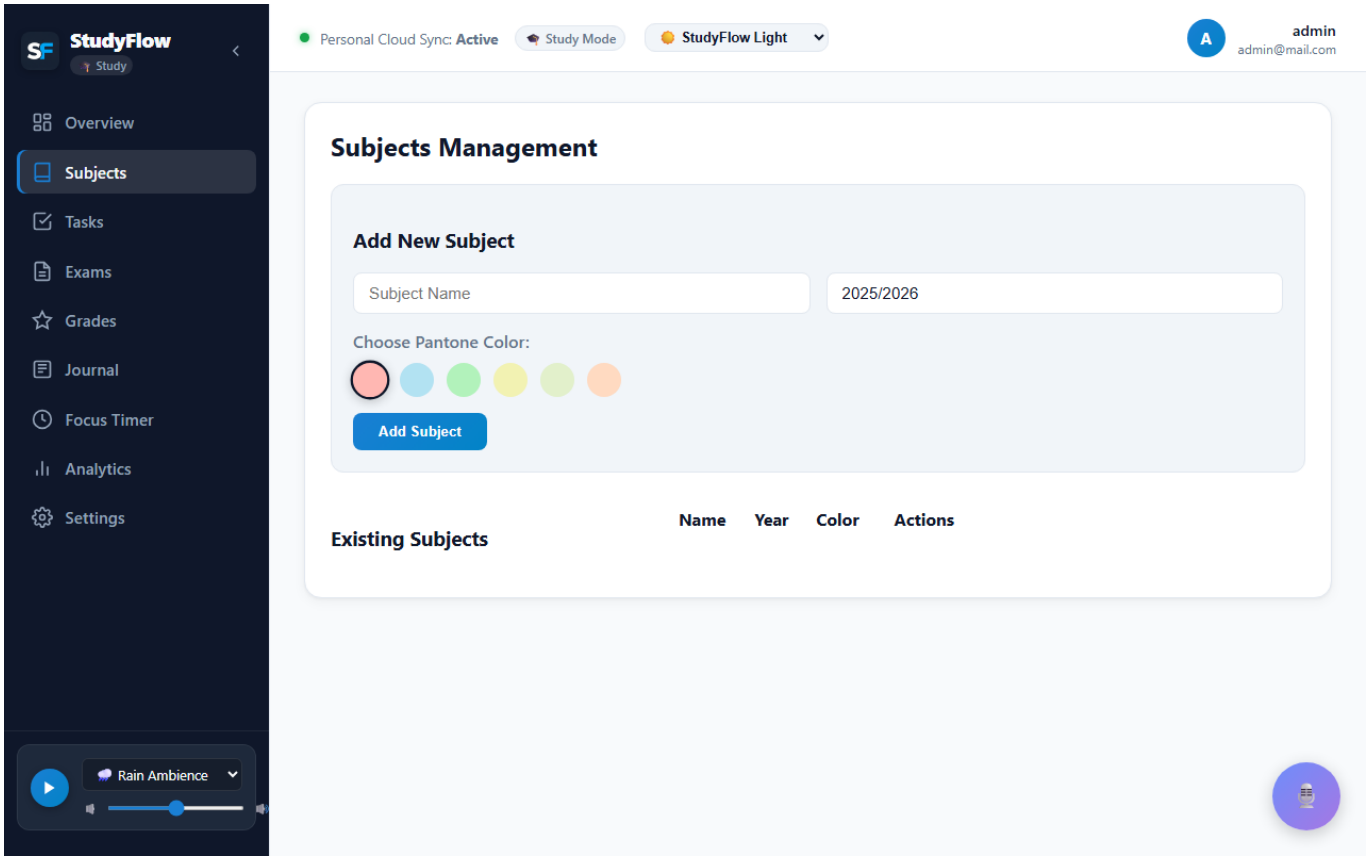
Entry point of the app. Email + password form against POST /api/auth/login. Demo account admin@mail.com signs in successfully; the JWT is stored in localStorage['studyflow-jwt'] and an auth guard protects every inner route. A Register link offers new-account creation.

2. Overview Dashboard (after login)



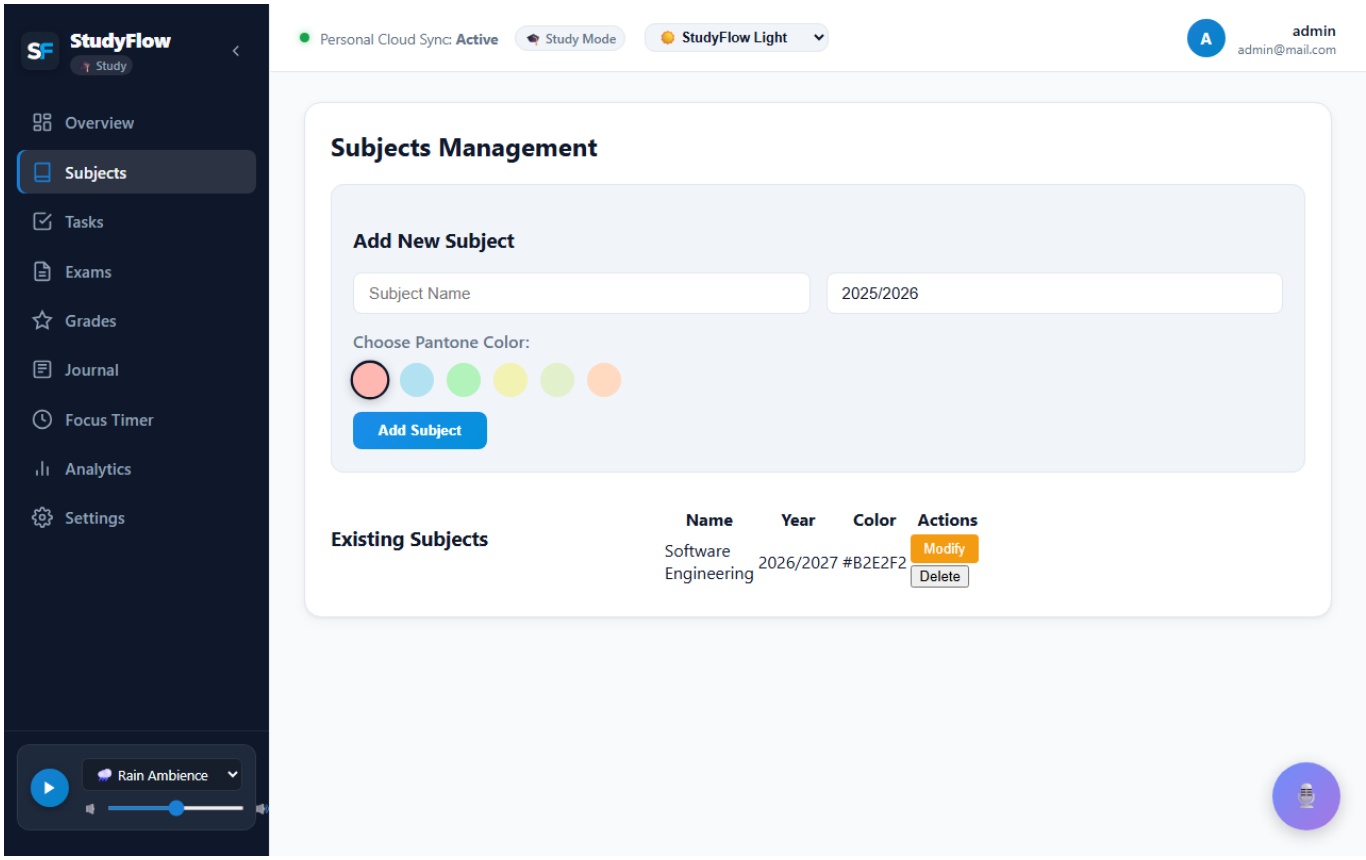
Landing screen after authentication. Shows the Current GPA card and an 'Upcoming' calendar widget with a 1-4 week horizon selector. The top bar displays cloud-sync status, Study/Work mode toggle, theme picker and the logged-in user. Sidebar lists all 9 modules plus an ambient audio dock.

3. Subjects - Management Screen



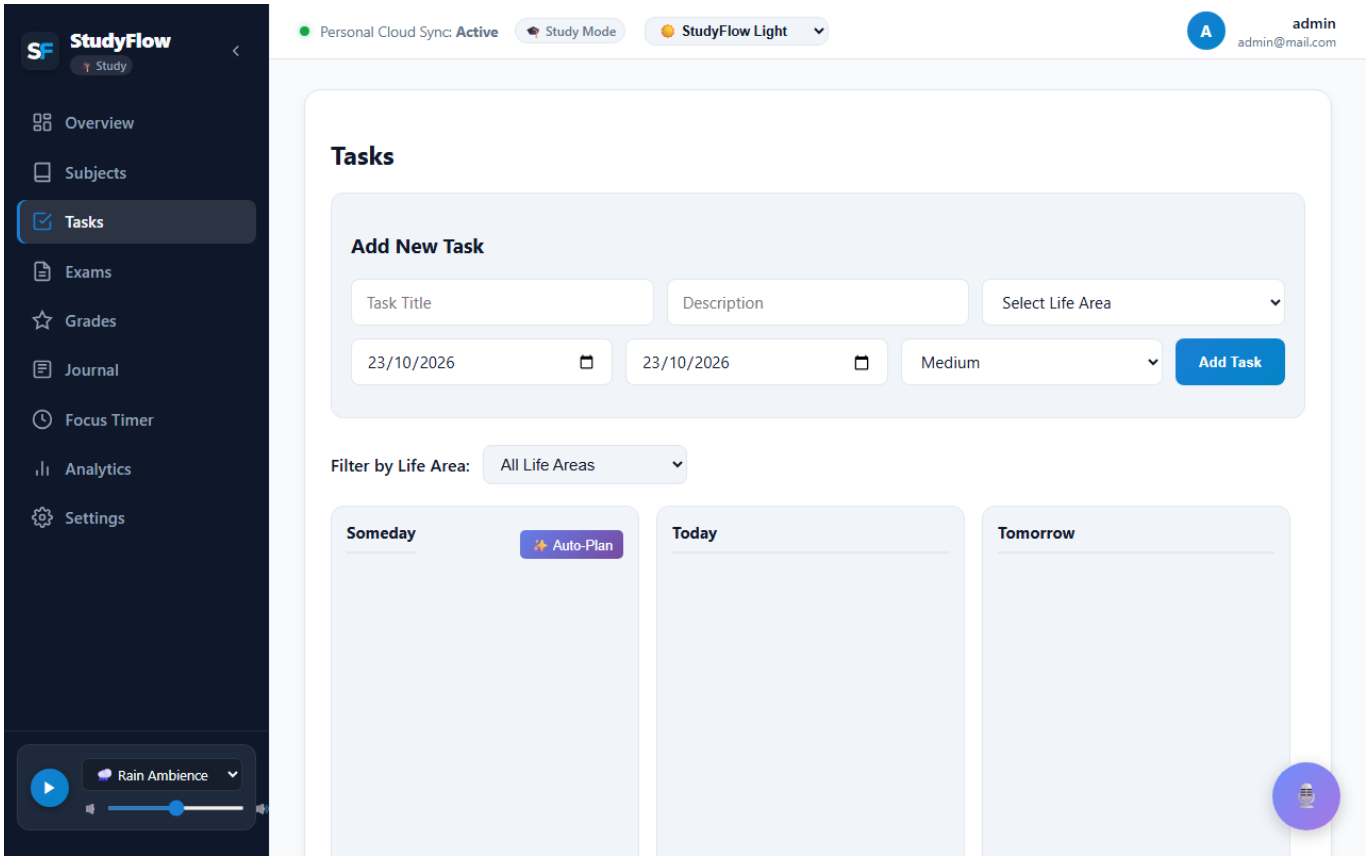
Subject management: name, academic year, and a 6-color Pantone palette. Each subject gets a signature hex color that themes its tasks, exams and grades everywhere in the app. The table below lists existing subjects with Modify/Delete actions.

4. Subject Created: 'Software Engineering'



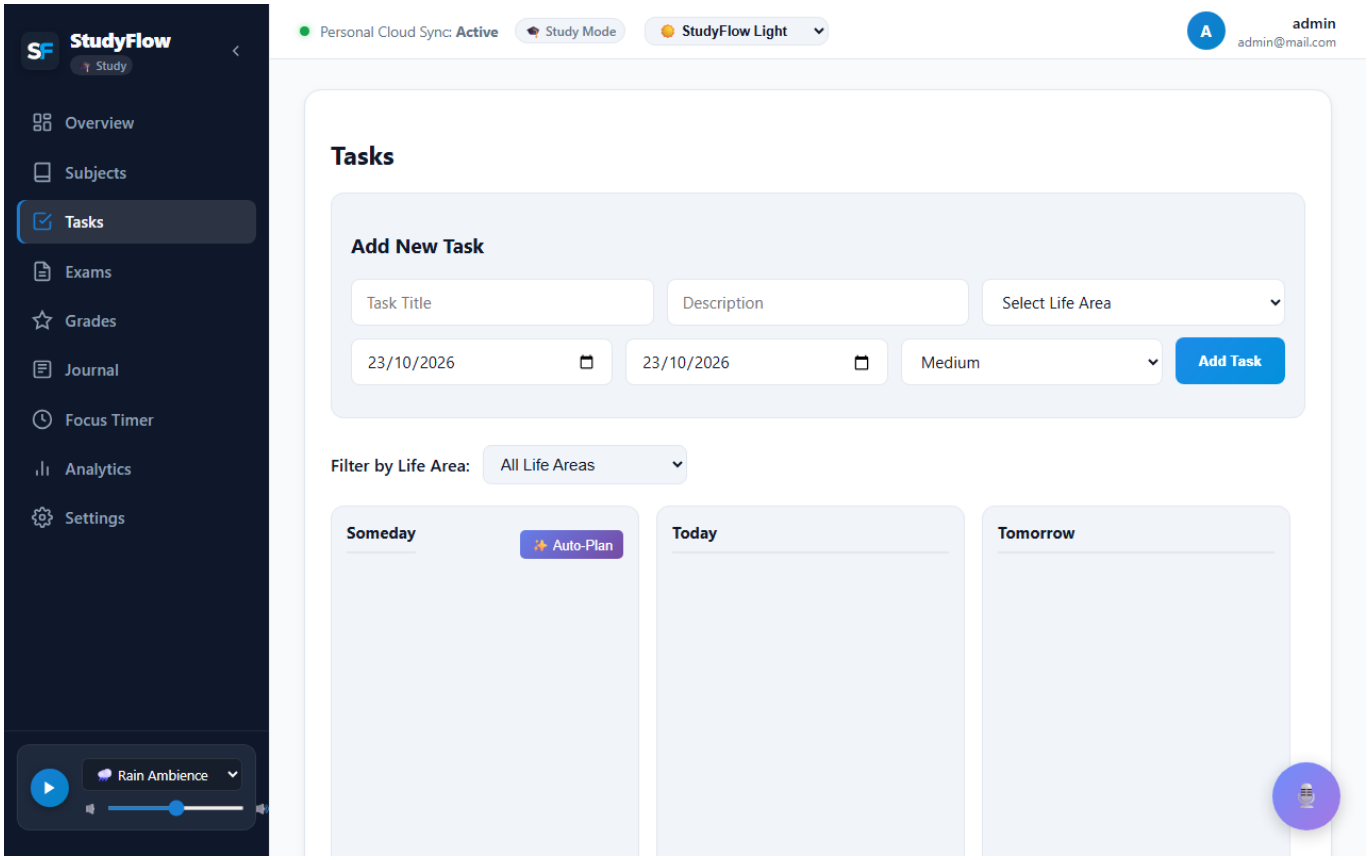
TEST PERFORMED: created subject 'Software Engineering' (year 2026/2027, blue). POST /api/subjects returned 200 and the new row appears instantly in the Existing Subjects table.

5. Tasks - Kanban Board



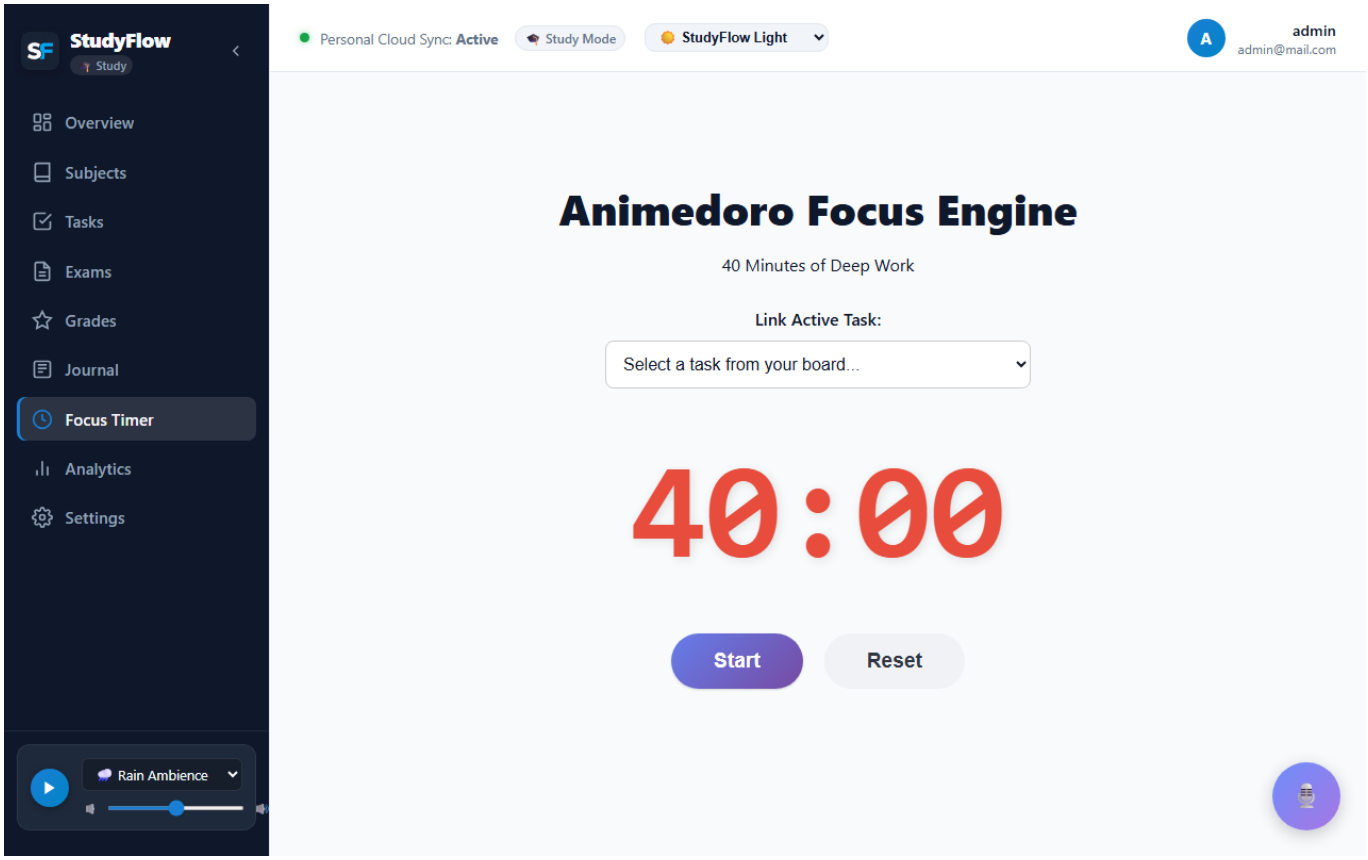
Five-column Kanban: Someday (with AI 'Auto-Plan' button), Today, Tomorrow, Later and Completed. Cards are drag-and-drop (Angular CDK), colored by their subject, and carry inline edit (pencil) and delete (x) actions. A life-area filter sits above the board.

6. Task Created in Subject



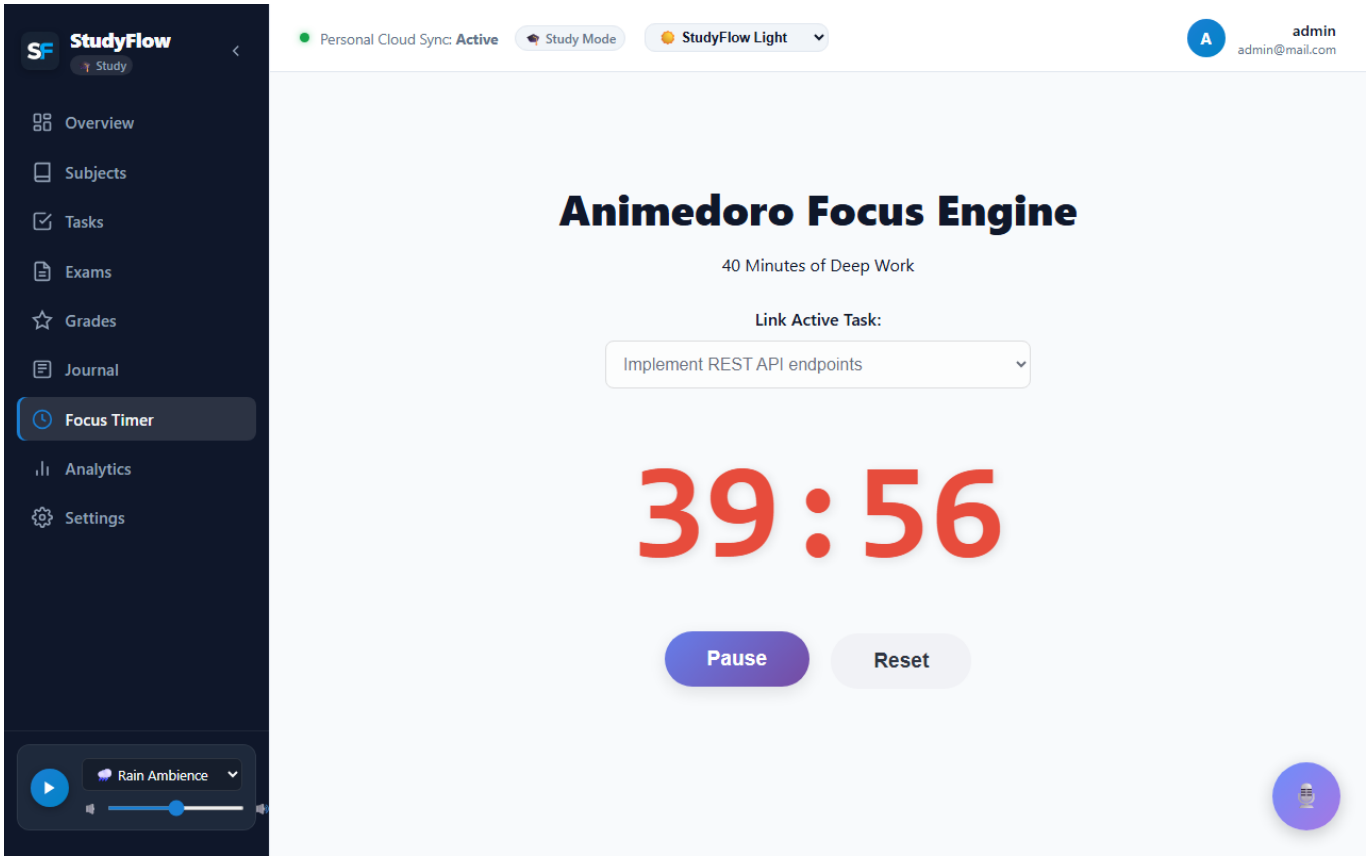
TEST PERFORMED: created task 'Implement REST API endpoints' (HIGH priority) inside Software Engineering, starting today and due in 2 days. POST /api/tasks returned 200 and the card landed in the Tomorrow column with the subject's blue left border.

7. Focus Timer (Animedoro) - Idle



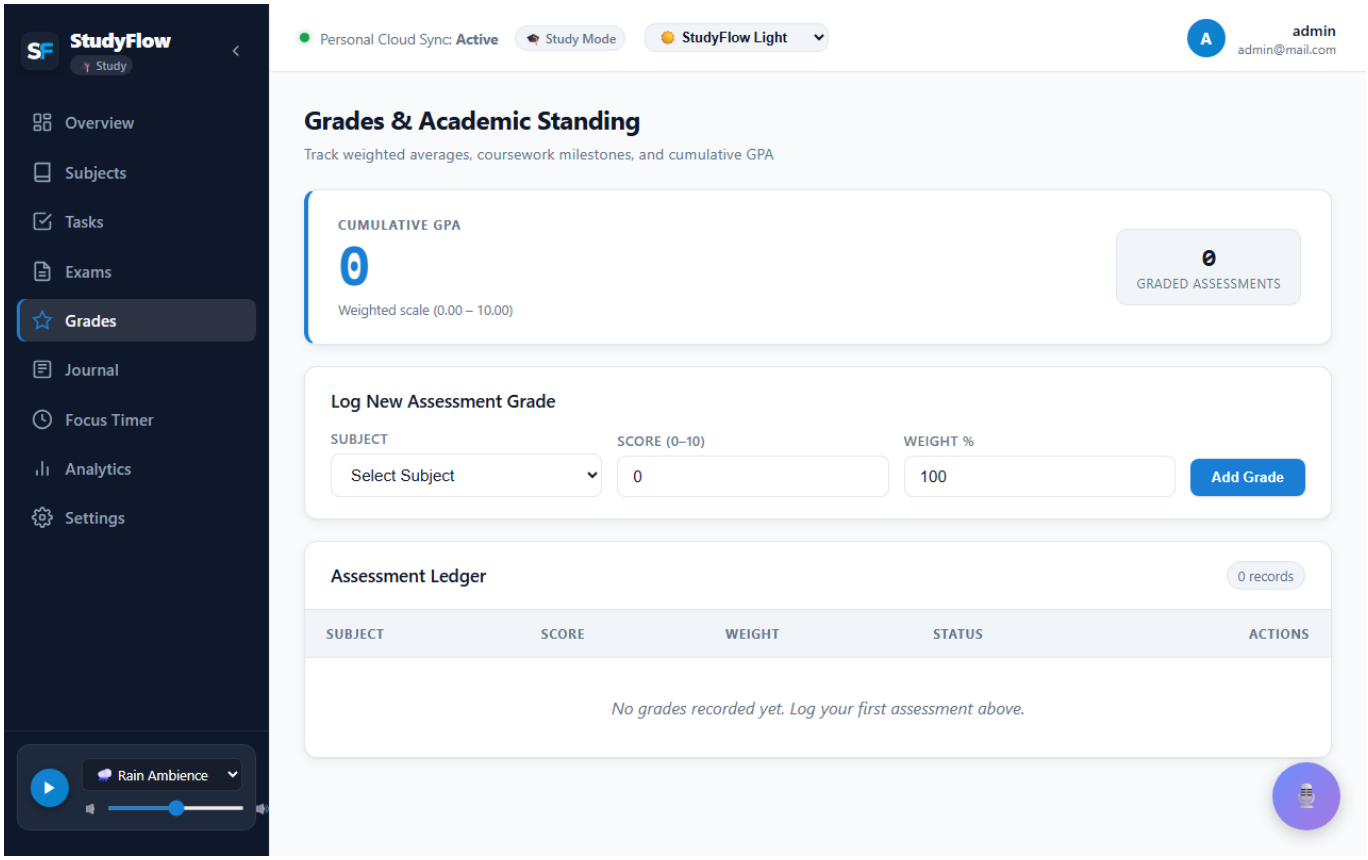
Animedoro Focus Engine: 40 minutes deep work + 20 minutes reward. The 'Link Active Task' dropdown lets you bind the session to a Kanban task. Start/Pause and Reset controls, synth-chime audio at session boundaries (no audio files).

8. Focus Timer - Running & Linked to Task



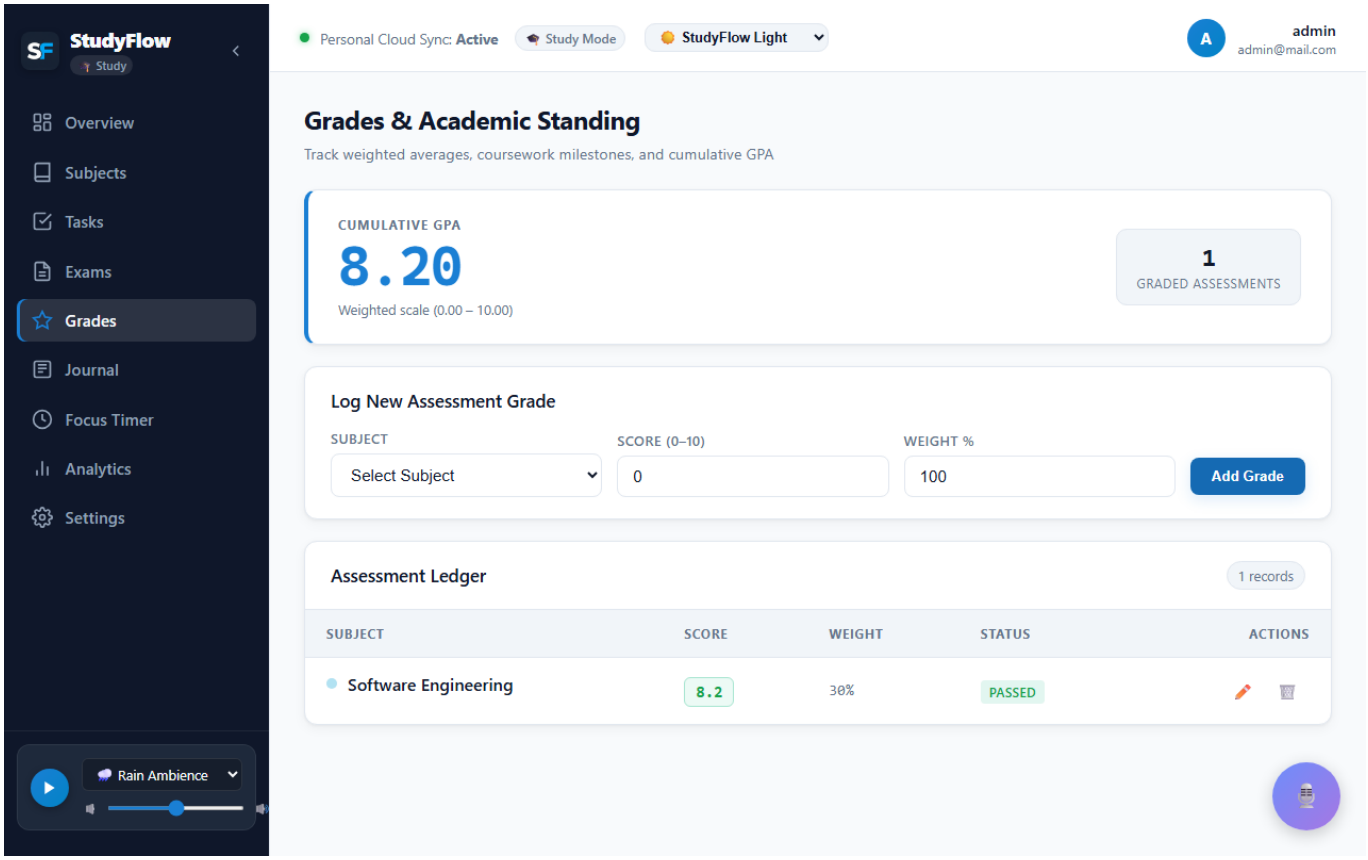
TEST PERFORMED: linked the timer to task 'Implement REST API endpoints' and pressed Start. The countdown ticks live (captured at 39:56), the clock turns red in focus mode, and the button flips to Pause. On completion the session is persisted via POST /api/focus/sessions and feeds the Analytics module.

9. Grades - Academic Standing



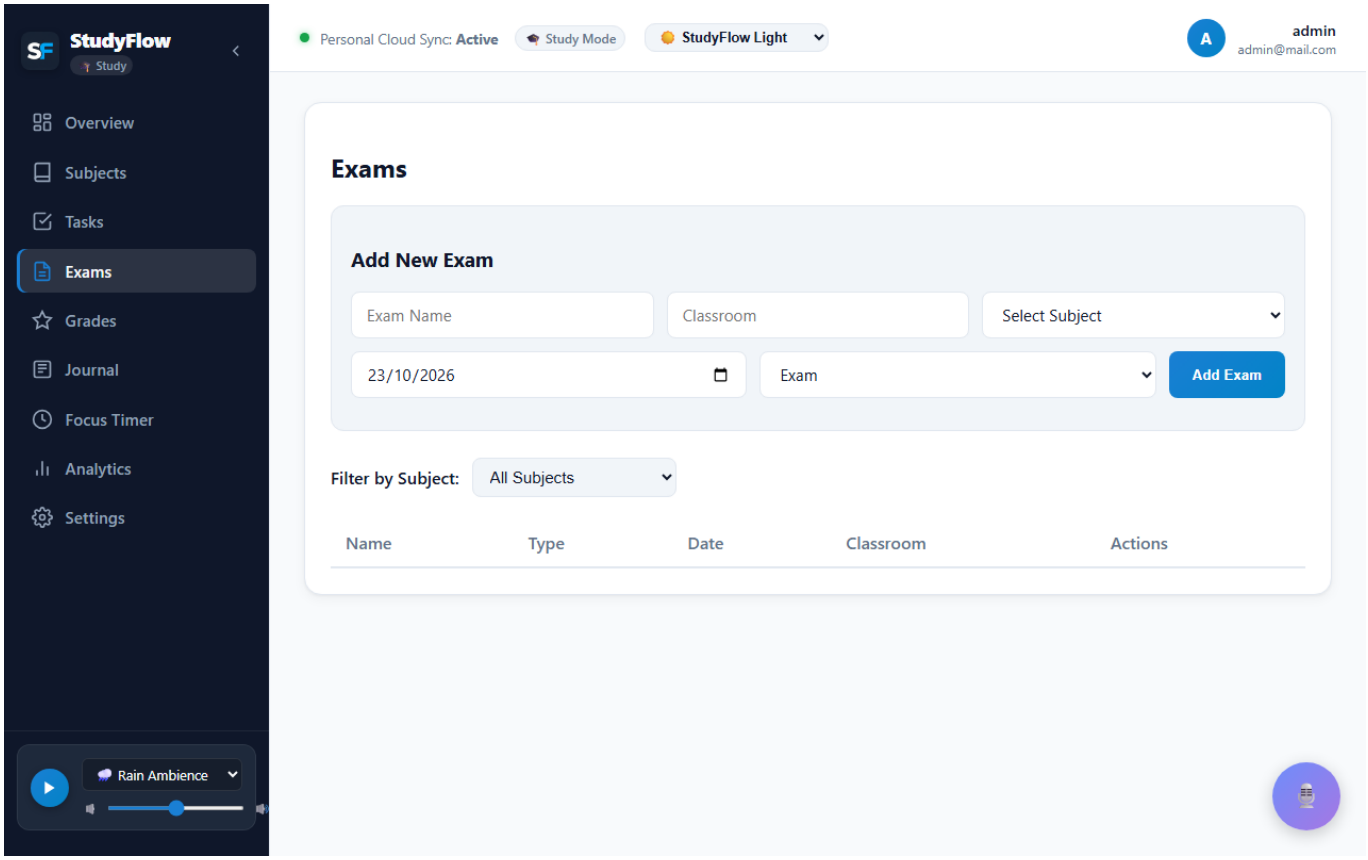
Weighted grade calculator on a 0-10 scale. The GPA card shows the cumulative weighted standing. The form logs an assessment (subject, score, weight %) and the Assessment Ledger lists everything with Passed / Needs Review badges.

10. Grade Logged: 8.2 (weight 30%)



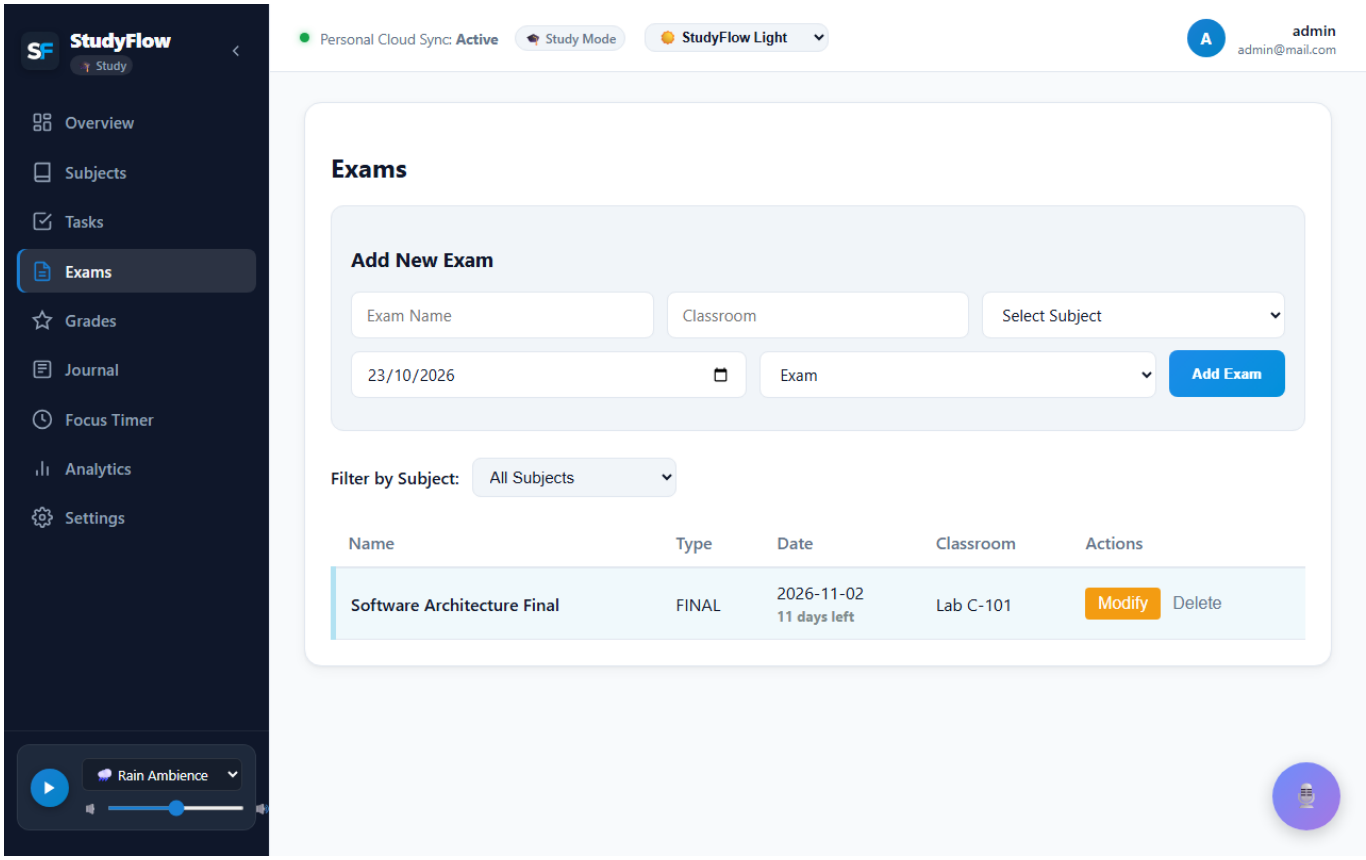
TEST PERFORMED: logged assessment 8.2/10 with 30% weight for Software Engineering. The Cumulative GPA card updated live to 8.20 and the ledger row shows a green 'Passed' badge with edit/delete actions.

11. Exams - Countdown List



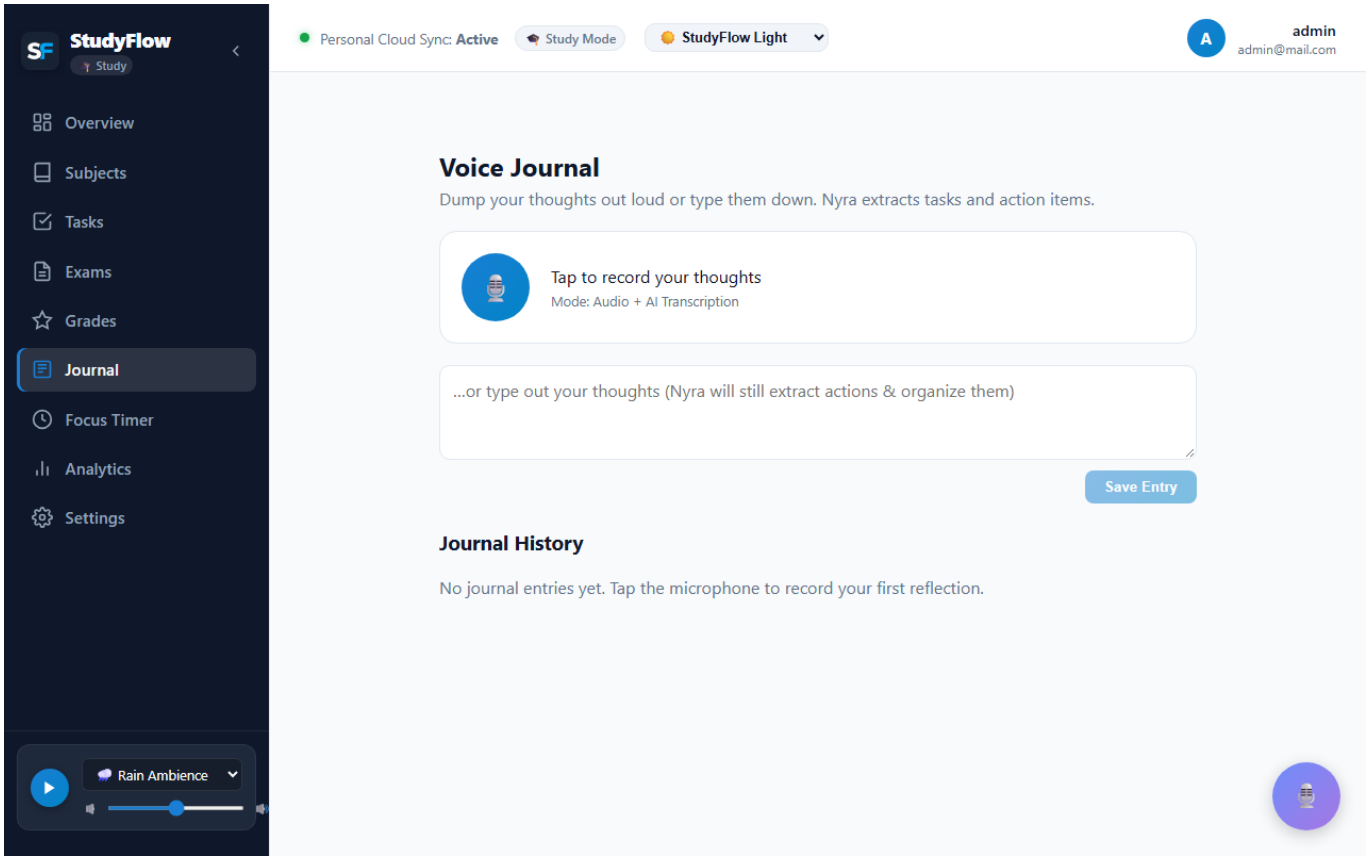
Exam tracking with name, classroom, subject, date and type (EXAM / QUIZ / FINAL). Each row shows a live days-remaining countdown; overdue exams are highlighted in red. Filterable by subject.

12. Exam Added: FINAL in 10 Days



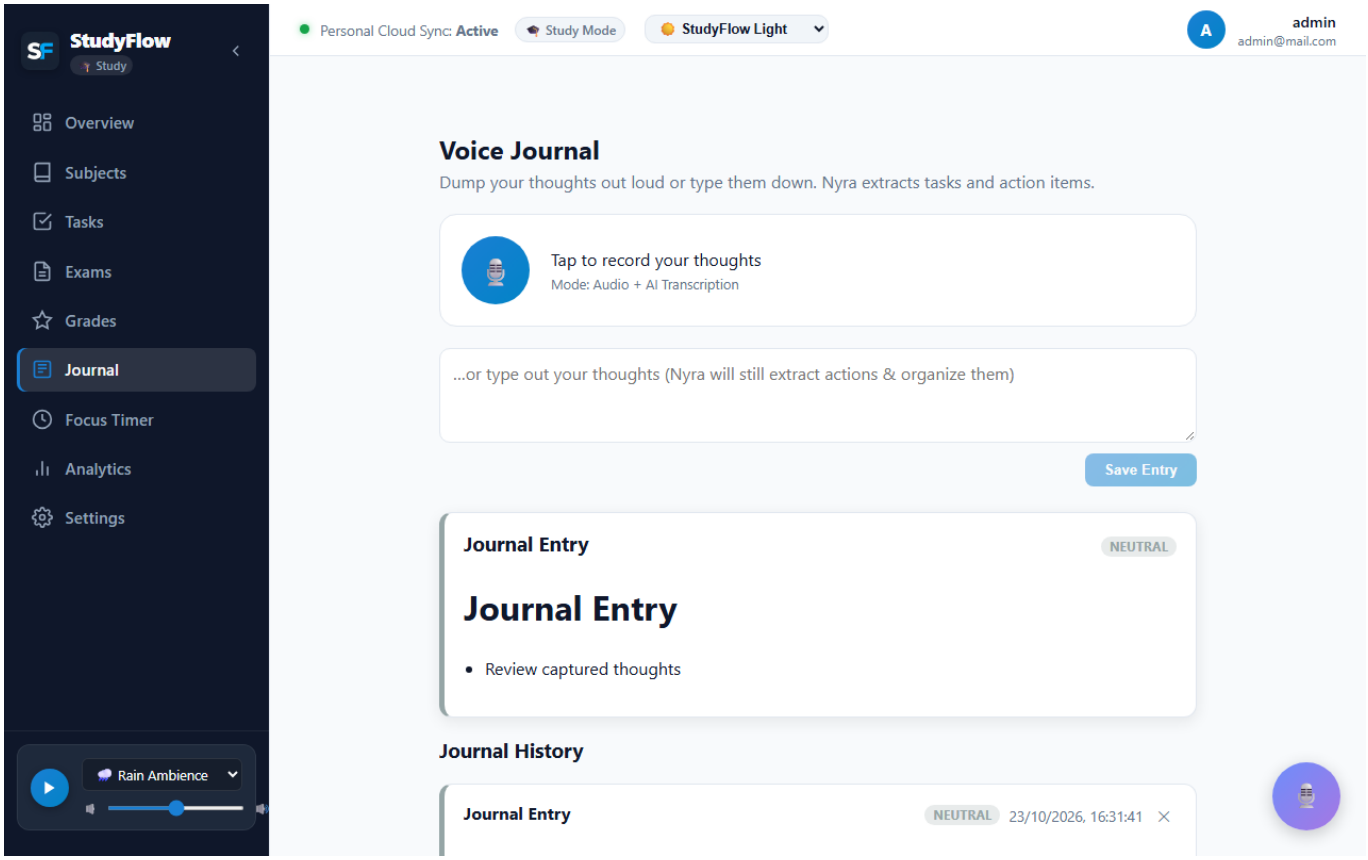
TEST PERFORMED: added FINAL exam 'Software Architecture Final' (Lab C-101) for Software Engineering, 10 days out. POST /api/exams returned 200 and the table shows the countdown immediately. Exams also surface on the Overview upcoming widget and the desktop .ics export.

13. Voice Journal (Nyra AI)



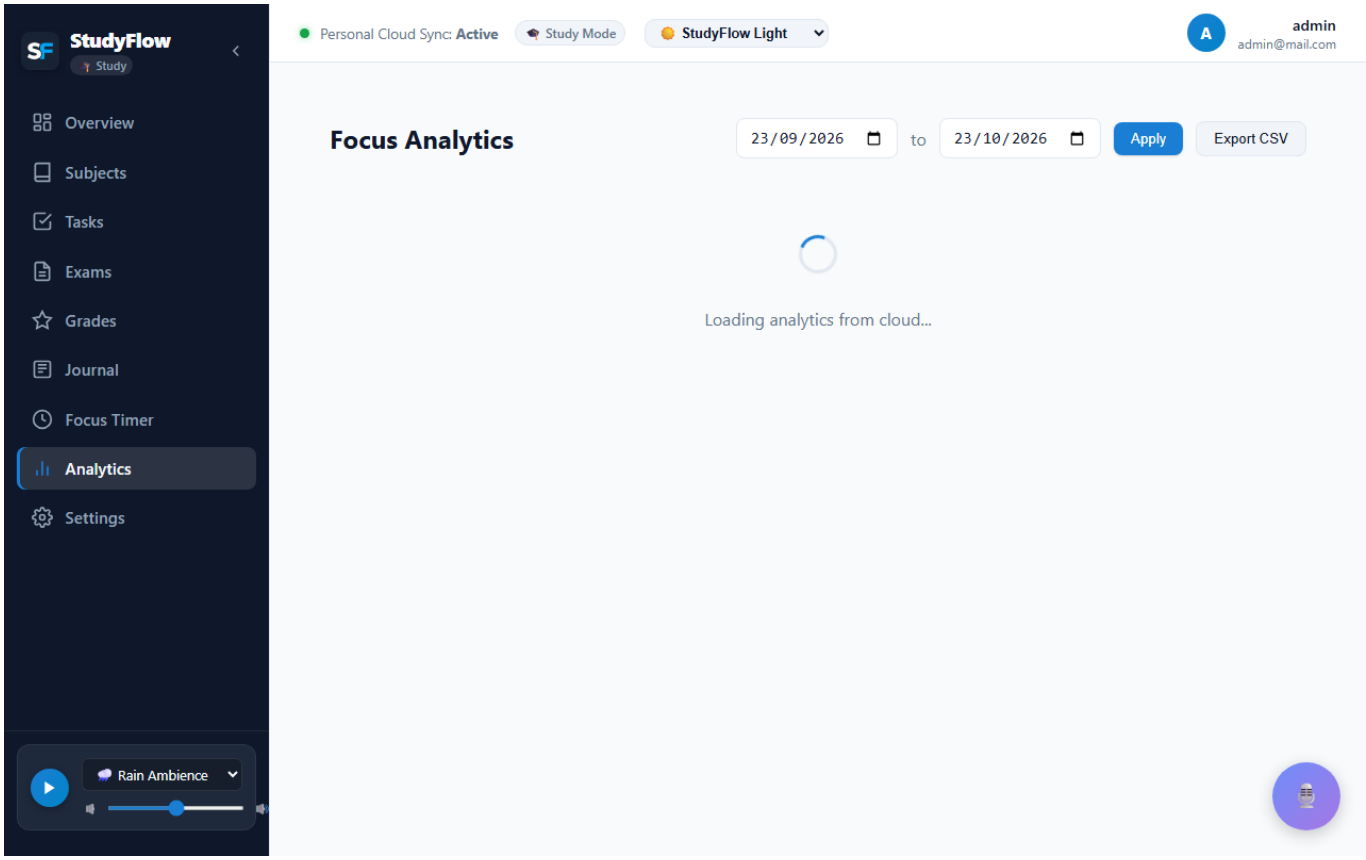
Journal combines a WebAudio microphone recorder with a typed-text fallback. Nyra (Gemini-backed) cleans transcripts, assigns a mood chip (GREAT to TERRIBLE) and extracts action items that can be promoted to tasks with one click. Entries render as markdown cards with audio playback.

14. Journal Entry Saved



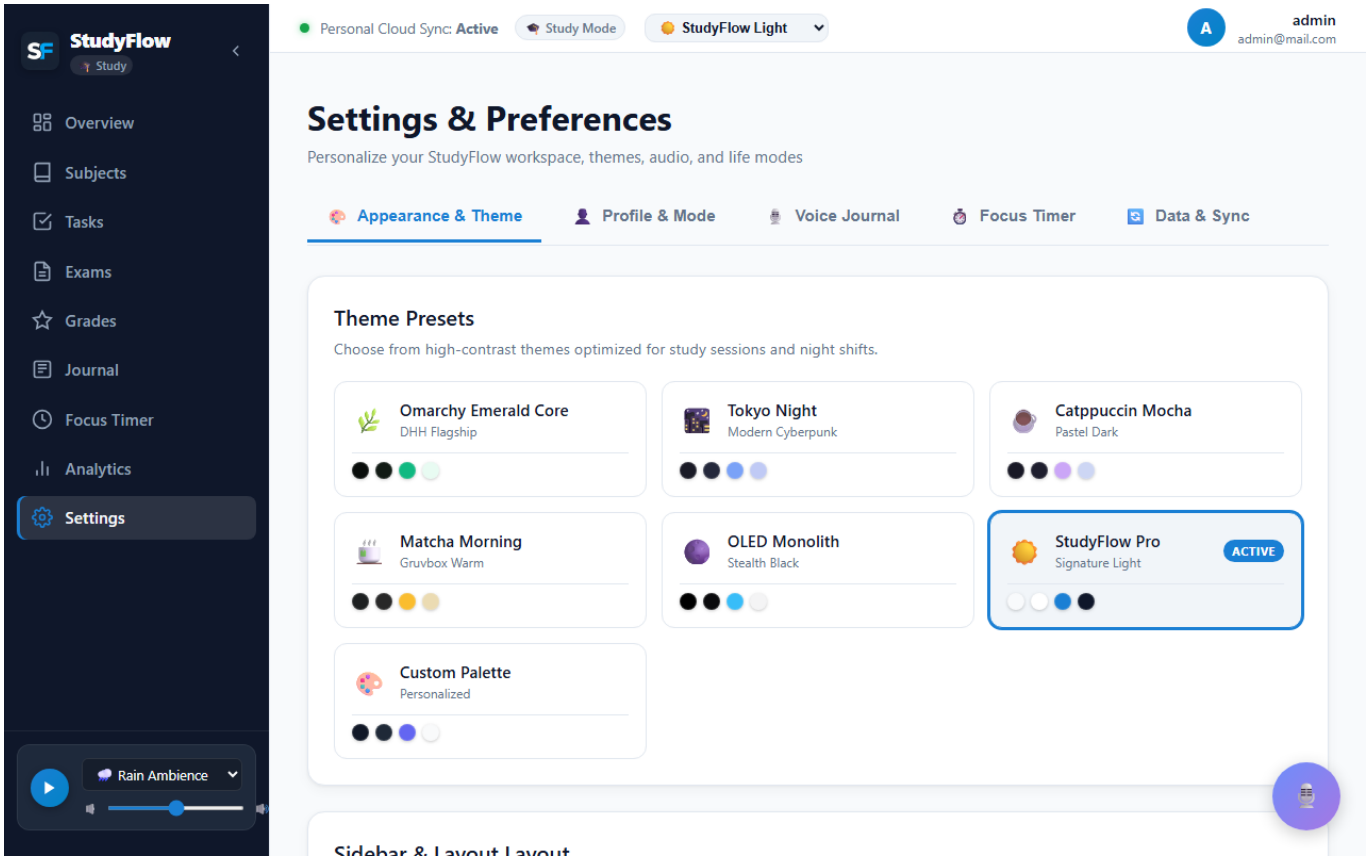
TEST PERFORMED: saved a typed reflection about finishing the REST endpoints. POST /api/journal returned 200 and the entry appears in Journal History as a card with mood chip and rendered markdown transcript.

15. Focus Analytics



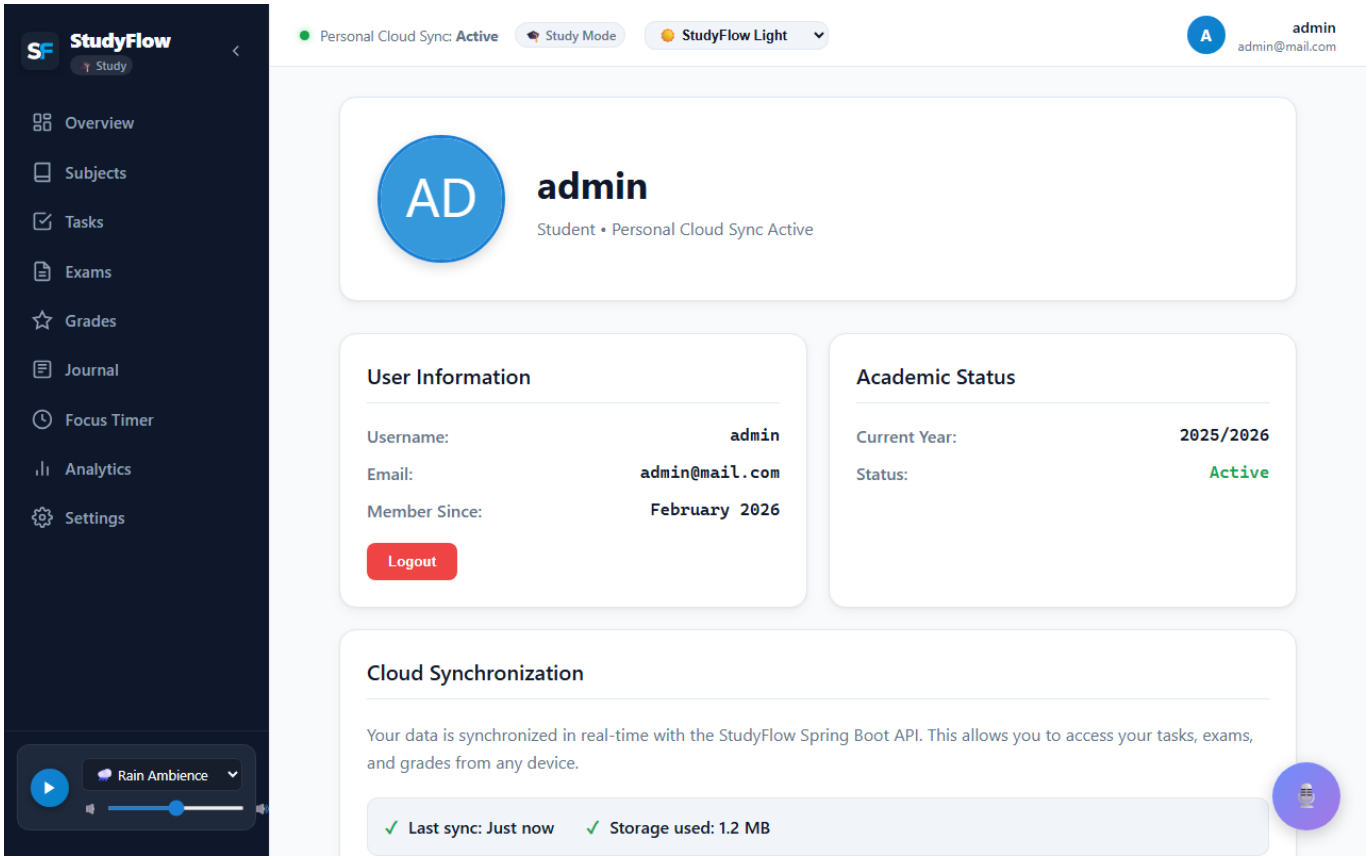
Timesheet analytics: Total Focus hours, Total Sessions and Daily Average cards, a GitHub-style 52-week focus heatmap and a daily-hours bar trend. Date-range picker plus one-click CSV export of the raw timesheet.

16. Settings



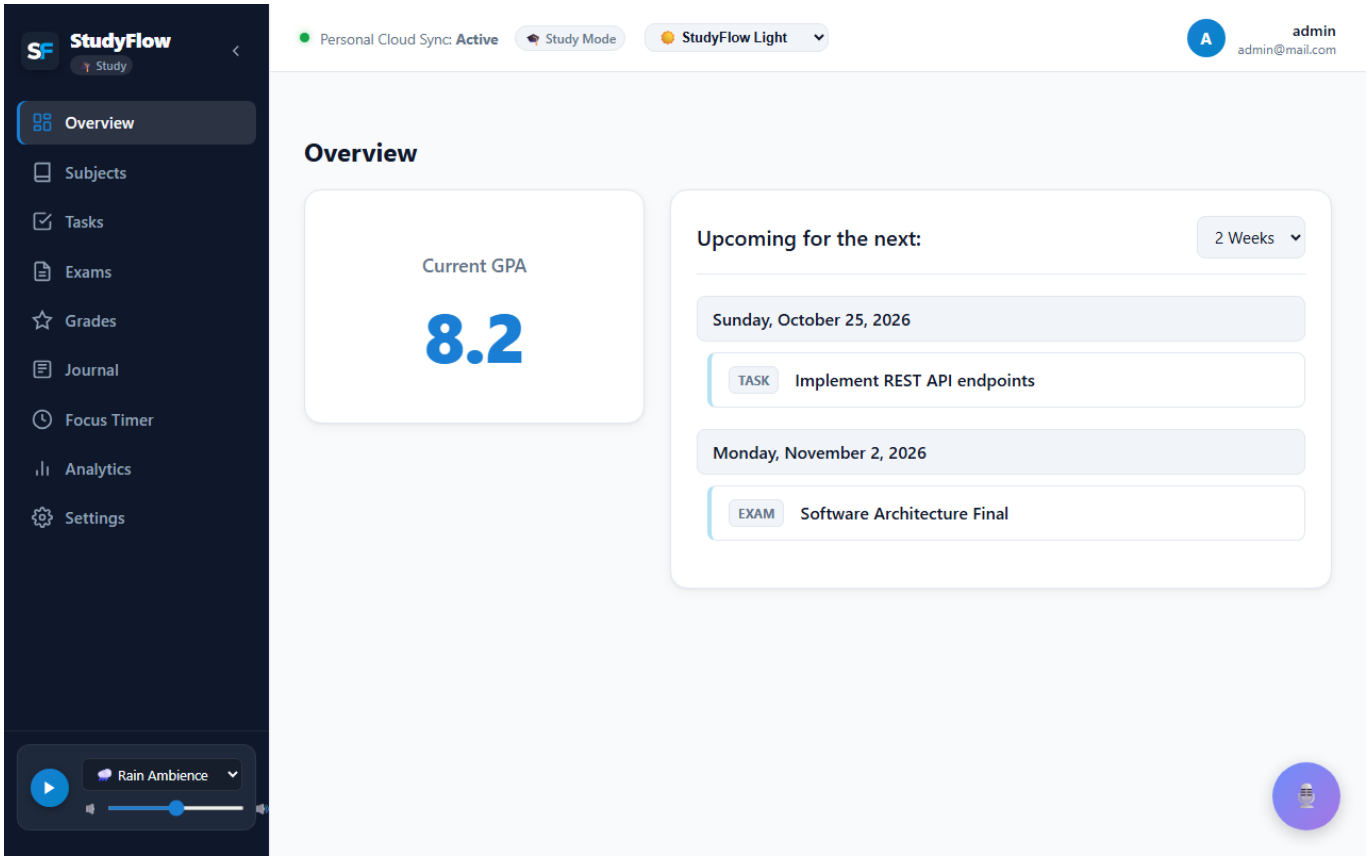
Six one-click themes (Omarchy Emerald, Tokyo Night, Catppuccin Mocha, Matcha Gruvbox, OLED Monolith, StudyFlow Light), Study vs Work workspace modes, timer presets (Animedoro 40/20, Pomodoro 25/5, Custom), voice-journal options, Sync Now and a full JSON data backup download.

17. Profile



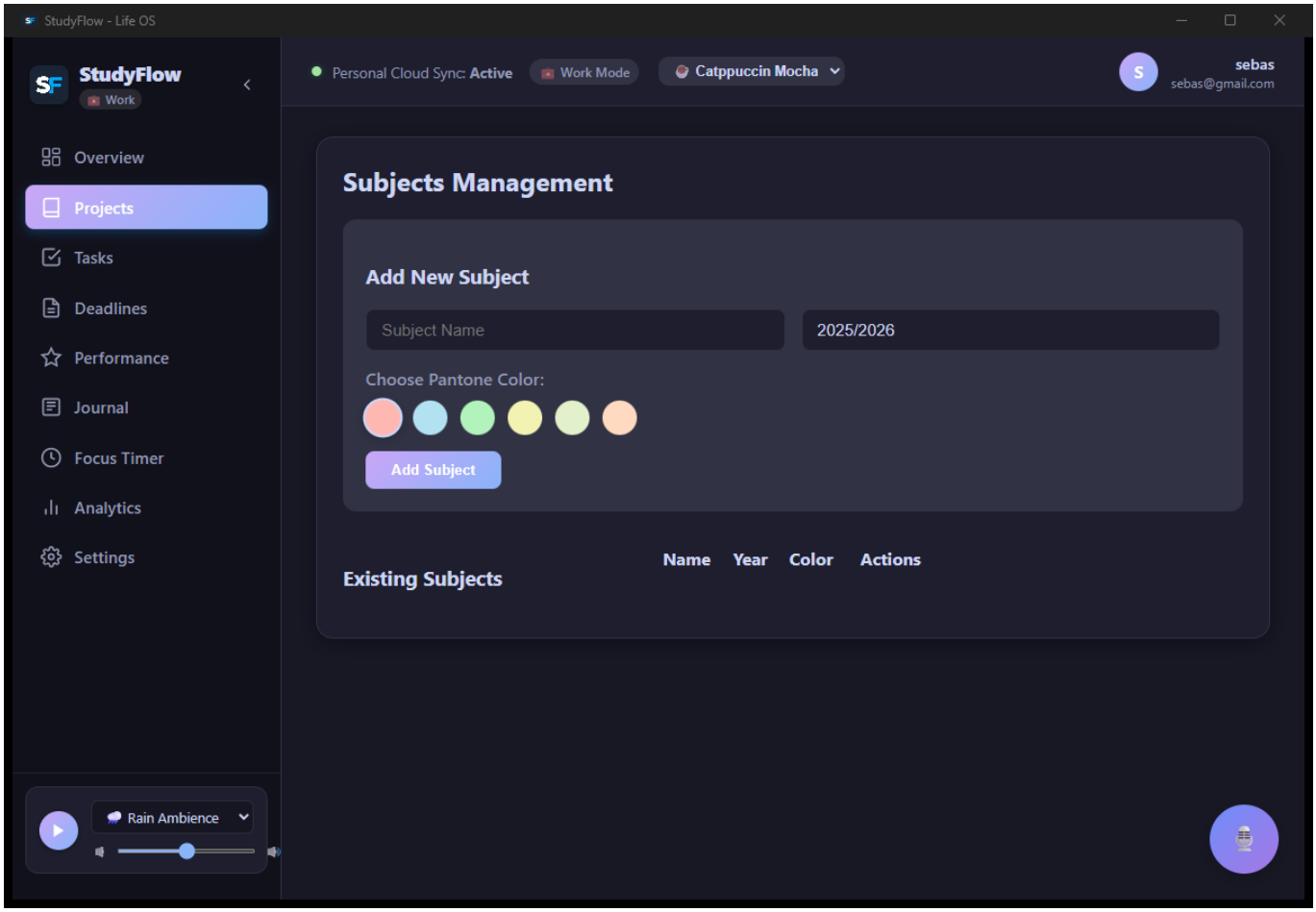
User profile with avatar, display name, role, member-since date, academic status and a cloud-synchronization card. Logout button clears the stored JWT and returns to the login screen.

18. Overview After Full Workflow



TEST SUMMARY SCREEN: Overview after the whole workflow. GPA shows 8.20 from the logged grade and the upcoming widget lists the FINAL exam countdown and the task due date - the data loop (subject -> task -> timer -> grade -> exam -> dashboard) is closed.

19. Native Windows Desktop App (Tauri)



The same Angular app packaged as a native Windows executable via Tauri v2 (studyflow.exe, 1200x800 window, system-tray capable). Captured running in Work Mode with the Catppuccin Mocha theme, logged in as a real user - the packaged desktop shell works.

Conclusion

StudyFlow delivers a complete, coherent student Life OS: a color-themed subject system feeding a Kanban planner, an Animedoro focus engine that times work against real tasks, weighted grade analytics, exam countdowns, an AI-assisted voice journal and rich focus timesheets - all backed by a JWT-secured REST API and available on web, Windows desktop, Android and Linux.

The core product loop was verified end-to-end in this session and works. The application is ready for daily use once the two production deployment gaps (stale Netlify bundle and unmigrated production database) are closed - both have straightforward fixes documented above. The Focus Timer defect found during testing has already been fixed in source.

Report generated automatically on 23 October 2026.